

Structures and STRUC

■ Defining Data Structures:

```
struc Person  
    .id:  resd 1  
    .name: resb 32  
    .age:  resd 1  
endstruc
```

■ Accessing: `mov eax, [rbx + Person.age]`

■ NASM structures are essentially just offsets. They provide a clean way to manage complex data memory offsets without manual math.

Revisiting swap

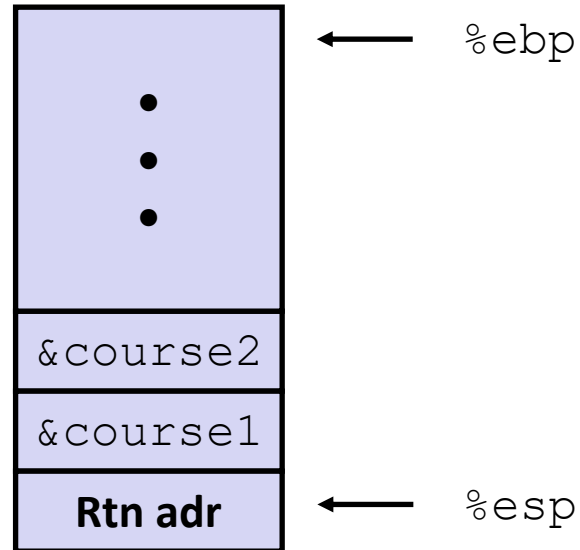
```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

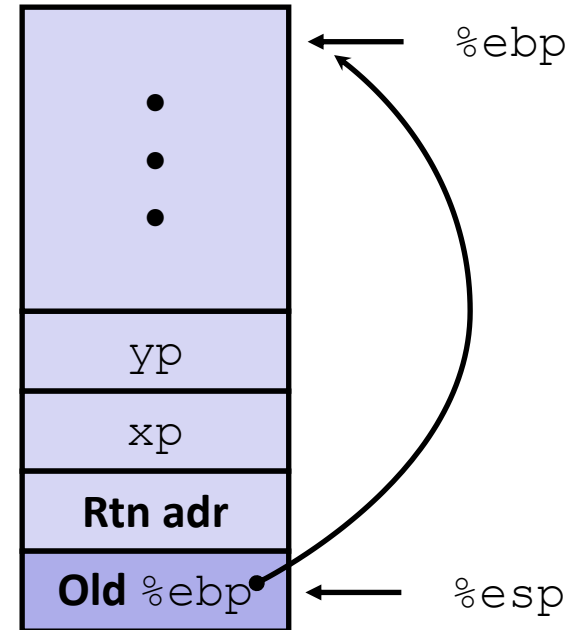
pushl %ebp	}	Set Up
movl %esp, %ebp		
pushl %ebx		
	}	Body
movl 8(%ebp), %edx		
movl 12(%ebp), %ecx		
movl (%edx), %ebx		
movl (%ecx), %eax		
movl %eax, (%edx)		
movl %ebx, (%ecx)		
	}	Finish
popl %ebx		
popl %ebp		
ret		

swap Setup #1

Entering Stack



Resulting Stack

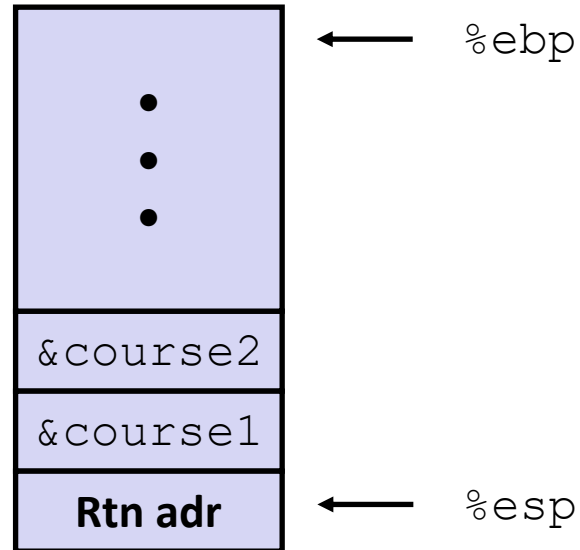


swap:

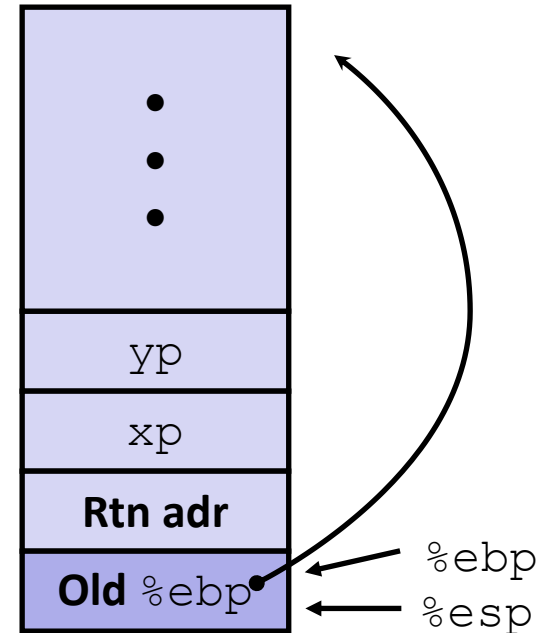
```
pushl %ebp
movl %esp, %ebp
pushl %ebx
```

swap Setup #2

Entering Stack



Resulting Stack

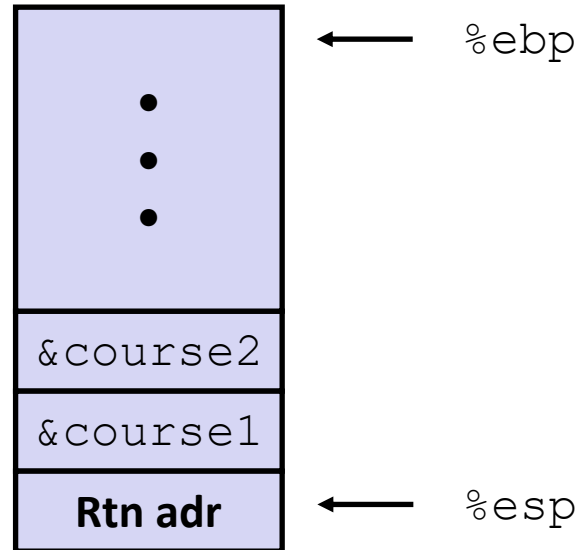


swap:

```
pushl %ebp
movl %esp, %ebp
pushl %ebx
```

swap Setup #3

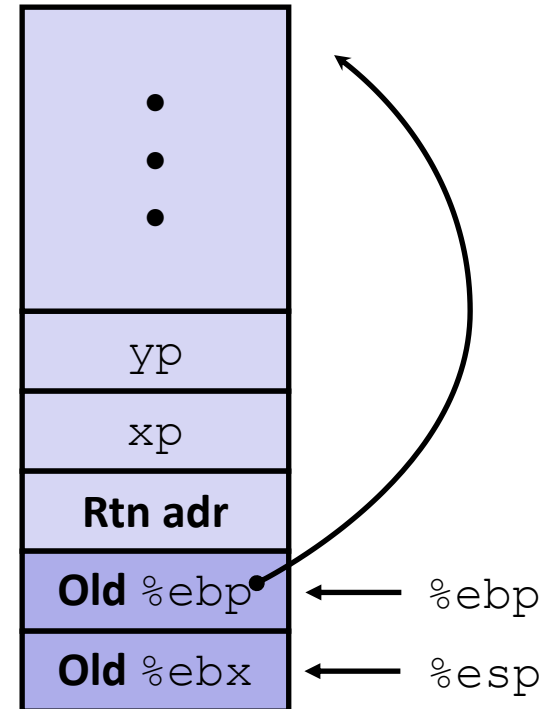
Entering Stack



swap:

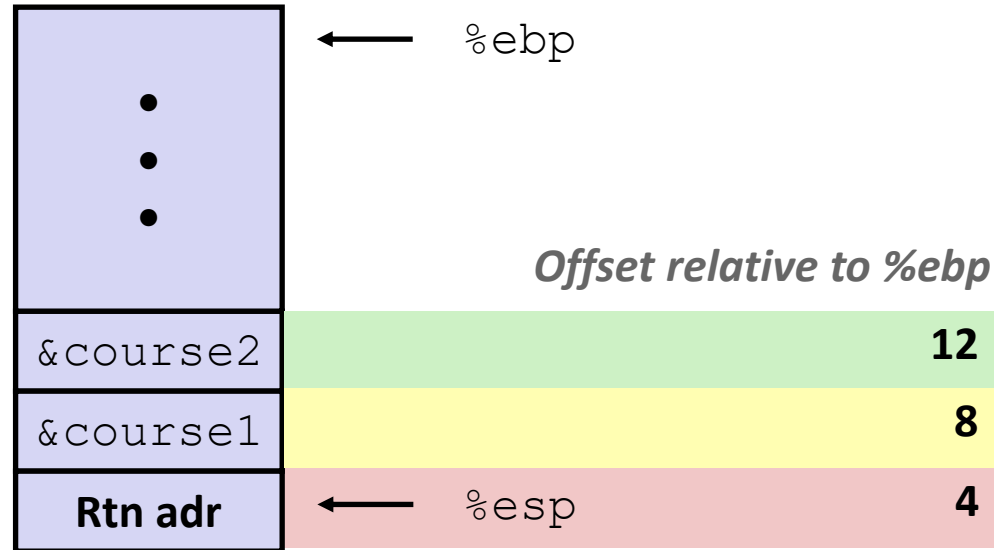
```
pushl %ebp  
movl %esp,%ebp  
pushl %ebx
```

Resulting Stack

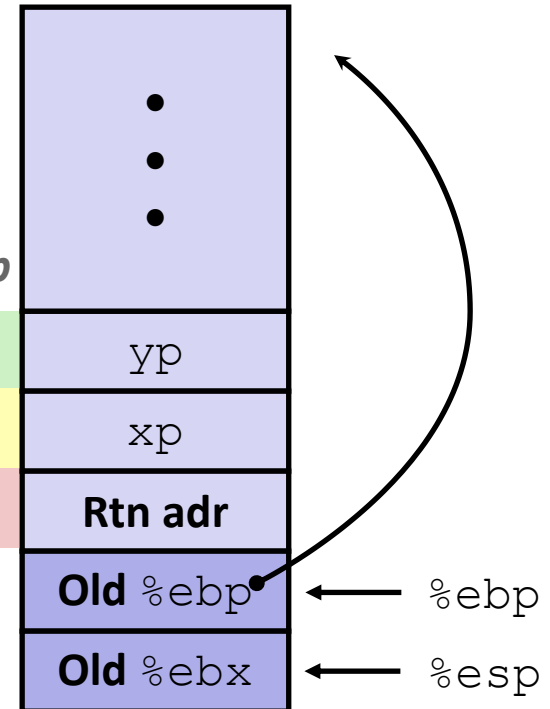


swap Body

Entering Stack



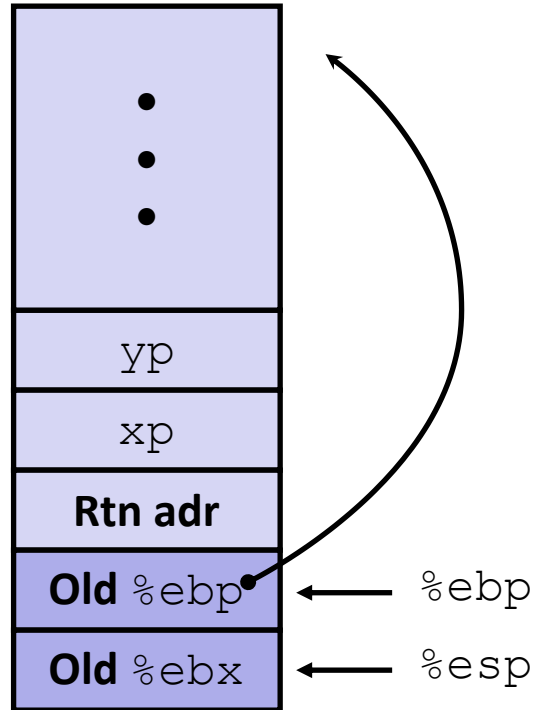
Resulting Stack



```
movl 8(%ebp),%edx    # get xp
movl 12(%ebp),%ecx   # get yp
. . .
```

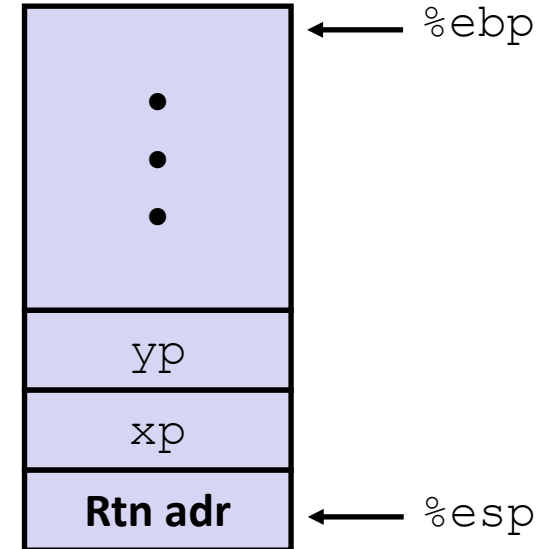
swap Finish

Stack Before Finish



popl %ebx
popl %ebp

Resulting Stack



■ Observation

- Saved and restored register %ebx
- Not so for %eax, %ecx, %edx

Disassembled swap

08048384 <swap>:

8048384:	55	push	%ebp
8048385:	89 e5	mov	%esp, %ebp
8048387:	53	push	%ebx
8048388:	8b 55 08	mov	0x8(%ebp), %edx
804838b:	8b 4d 0c	mov	0xc(%ebp), %ecx
804838e:	8b 1a	mov	(%edx), %ebx
8048390:	8b 01	mov	(%ecx), %eax
8048392:	89 02	mov	%eax, (%edx)
8048394:	89 19	mov	%ebx, (%ecx)
8048396:	5b	pop	%ebx
8048397:	5d	pop	%ebp
8048398:	c3	ret	

Calling Code

80483b4:	movl	\$0x8049658, 0x4(%esp)	# Copy &course2
80483bc:	movl	\$0x8049654, (%esp)	# Copy &course1
80483c3:	call	8048384 <swap>	# Call swap
80483c8:	leave		# Prepare to return
80483c9:	ret		# Return

Register Saving Conventions

■ When procedure `yoo` calls `who`:

- `yoo` is the *caller*
- `who` is the *callee*

■ Can register be used for temporary storage?

```
yoo:
    . . .
    movl $15213, %edx
    call who
    addl %edx, %eax
    . . .
    ret
```

```
who:
    . . .
    movl 8(%ebp), %edx
    addl $18243, %edx
    . . .
    ret
```

- Contents of register `%edx` overwritten by `who`
- This could be trouble → something should be done!
 - Need some coordination

Register Saving Conventions

■ When procedure `yoo` calls `who`:

- `yoo` is the *caller*
- `who` is the *callee*

■ Can register be used for temporary storage?

■ Conventions

- *“Caller Save”*
 - Caller saves temporary values in its frame before the call
- *“Callee Save”*
 - Callee saves temporary values in its frame before using

IA32/Linux+Windows Register Usage

■ **%eax, %edx, %ecx**

- Caller saves prior to call if values are used later

■ **%eax**

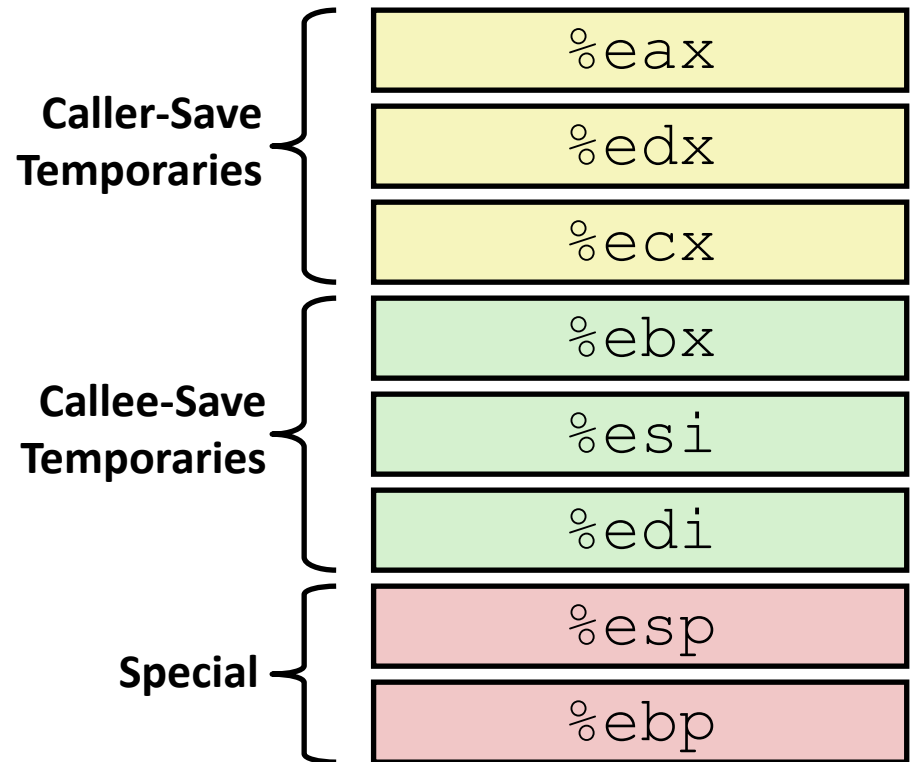
- also used to return integer value

■ **%ebx, %esi, %edi**

- Callee saves if wants to use them

■ **%esp, %ebp**

- special form of callee save
- Restored to original values upon exit from procedure



Recursive Function

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

■ Registers

- `%eax, %edx` used without first saving
- `%ebx` used, but saved at beginning & restored at end

```
pcount_r:
    pushl %ebp
    movl %esp, %ebp
    pushl %ebx
    subl $4, %esp
    movl 8(%ebp), %ebx
    movl $0, %eax
    testl %ebx, %ebx
    je .L3
    movl %ebx, %eax
    shrl %eax
    movl %eax, (%esp)
    call pcount_r
    movl %ebx, %edx
    andl $1, %edx
    leal (%edx, %eax), %eax
.L3:
    addl $4, %esp
    popl %ebx
    popl %ebp
    ret
```

Recursive Call #1

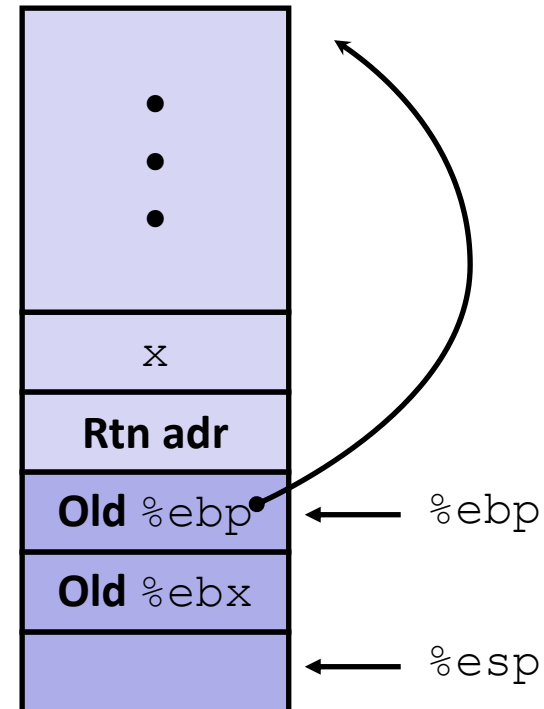
```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

■ Actions

- Save old value of `%ebx` on stack
- Allocate space for argument to recursive call
- Store `x` in `%ebx`



```
pcount_r:
    pushl %ebp
    movl %esp, %ebp
    pushl %ebx
    subl $4, %esp
    movl 8(%ebp), %ebx
    . . .
```



Recursive Call #2

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

```
    . . .
movl  $0, %eax
testl %ebx, %ebx
je    .L3
    . . .
.L3:
    . . .
ret
```

■ Actions

- If `x == 0`, return
 - with `%eax` set to 0

`%ebx`

`x`

Recursive Call #3

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

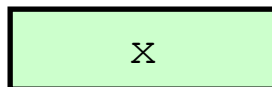
■ Actions

- Store $x \gg 1$ on stack
- Make recursive call

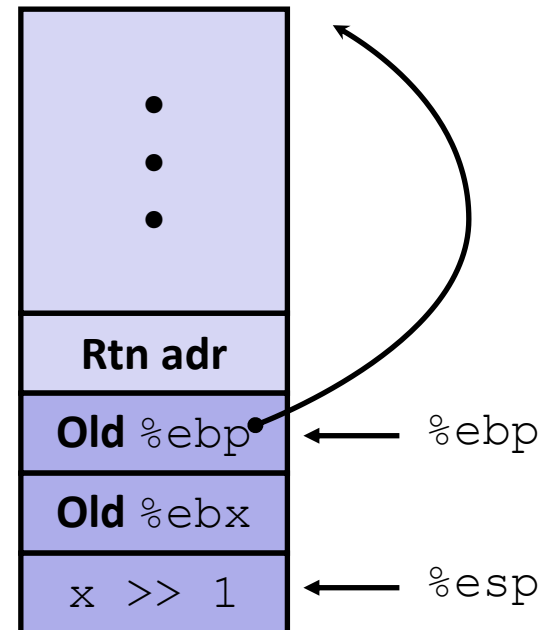
■ Effect

- `%eax` set to function result
- `%ebx` still has value of x

`%ebx`



```
• • •
movl  %ebx, %eax
shrl  %eax
movl  %eax, (%esp)
call  pcount_r
• • •
```



Recursive Call #4

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

```
• • •
movl    %ebx, %edx
andl    $1, %edx
leal    (%edx,%eax), %eax
• • •
```

■ Assume

- `%eax` holds value from recursive call
- `%ebx` holds `x`

■ Actions

- Compute `(x & 1) + computed value`

■ Effect

- `%eax` set to function result



Recursive Call #5

```
/* Recursive popcount */
int pcount_r(unsigned x) {
    if (x == 0)
        return 0;
    else return
        (x & 1) + pcount_r(x >> 1);
}
```

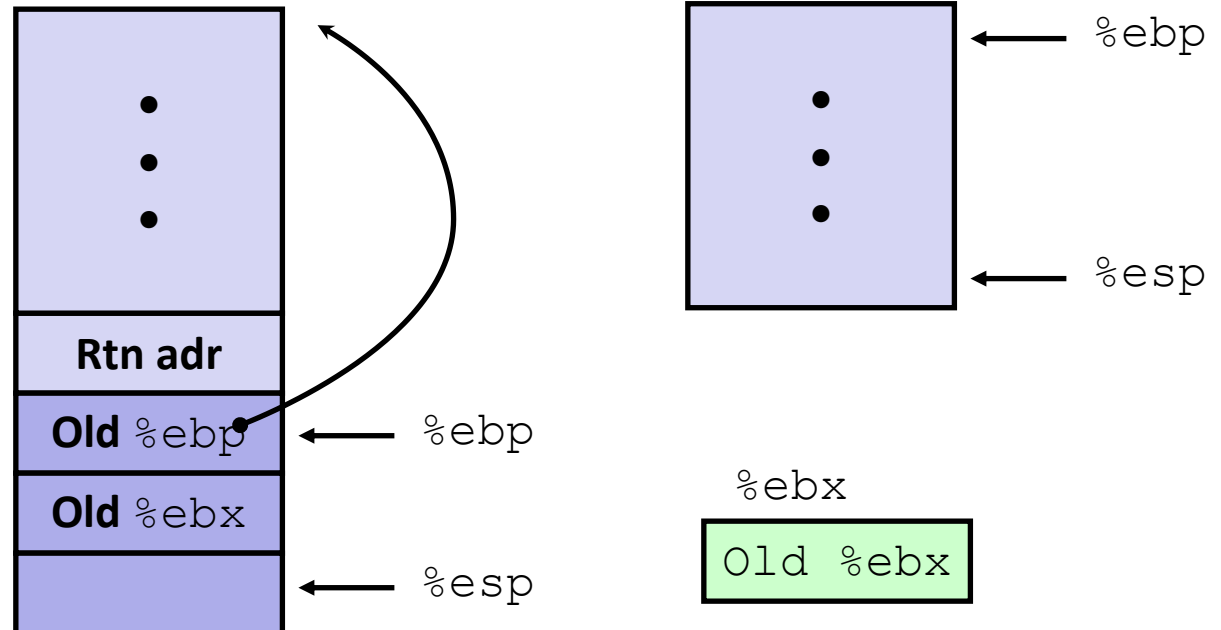
...

L3:

```
addl$4, %esp
popl%ebx
popl%ebp
ret
```

■ Actions

- Restore values of %ebx and %ebp
- Restore %esp



Observations About Recursion

■ Handled Without Special Consideration

- Stack frames mean that each function call has private storage
 - Saved registers & local variables
 - Saved return pointer
- Register saving conventions prevent one function call from corrupting another's data
- Stack discipline follows call / return pattern
 - If P calls Q, then Q returns before P
 - Last-In, First-Out

■ Also works for mutual recursion

- P calls Q; Q calls P

Pointer Code

Generating Pointer

```
/* Compute x + 3 */
int add3(int x) {
    int localx = x;
    incrk(&localx, 3);
    return localx;
}
```

Referencing Pointer

```
/* Increment value by k */
void incrk(int *ip, int k) {
    *ip += k;
}
```

- **add3** creates pointer and passes it to **incrk**

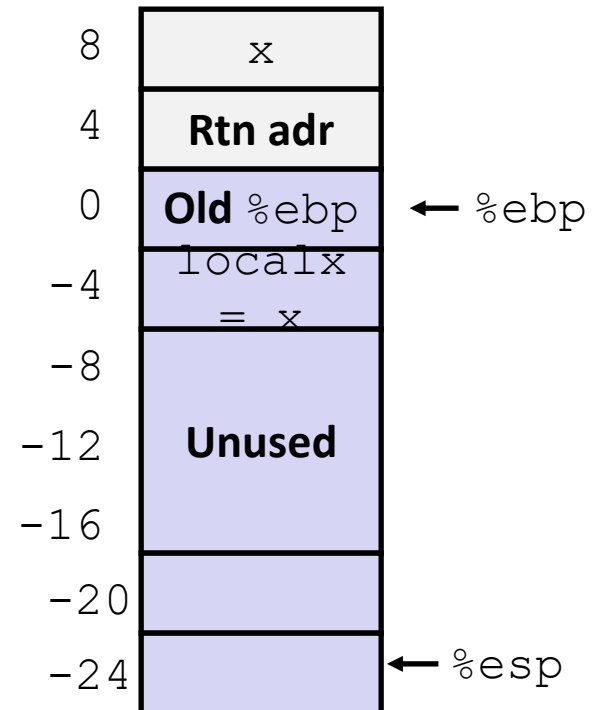
Creating and Initializing Local Variable

```
int add3(int x) {  
    int localx = x;  
    incr(&localx, 3);  
    return localx;  
}
```

- Variable localx must be stored on stack
 - Because: Need to create pointer to it
- Compute pointer as -4(%ebp)

First part of add3

```
add3:  
    pushl %ebp  
    movl  %esp, %ebp  
    subl  $24, %esp      # Alloc. 24 bytes  
    movl  8(%ebp), %eax  
    movl  %eax, -4(%ebp) # Set localx to x
```



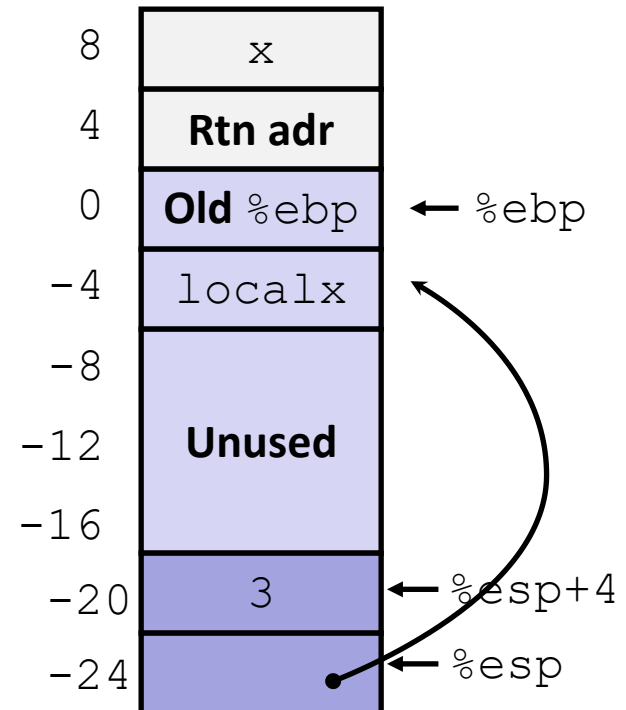
Creating Pointer as Argument

```
int add3(int x) {  
    int localx = x;  
    incrk(&localx, 3);  
    return localx;  
}
```

- Use leal instruction to compute address of localx

Middle part of add3

```
movl $3, 4(%esp)    # 2nd arg = 3  
leal -4(%ebp), %eax # &localx  
movl %eax, (%esp)   # 1st arg = &localx  
call incrk
```



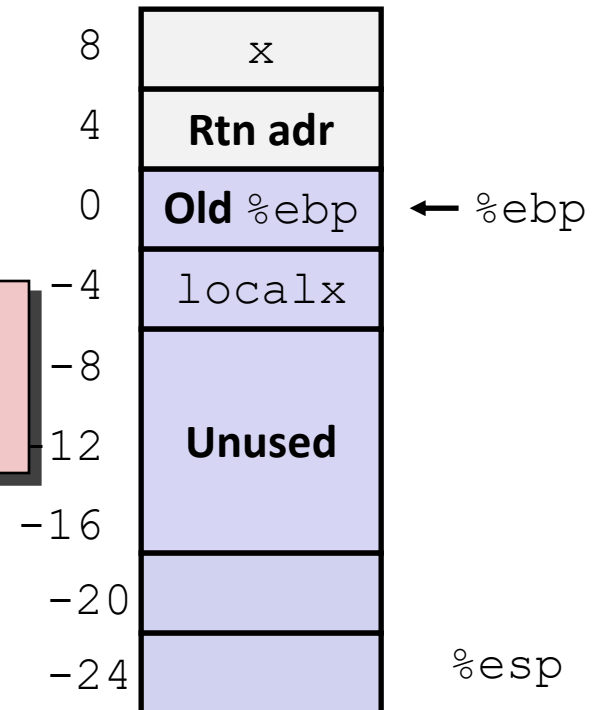
Retrieving local variable

```
int add3(int x) {  
    int localx = x;  
    incrk(&localx, 3);  
    return localx;  
}
```

- Retrieve localx from stack as return value

Final part of add3

```
movl -4(%ebp), %eax # Return val= localx  
leave  
ret
```



IA 32 Procedure Summary

■ Important Points

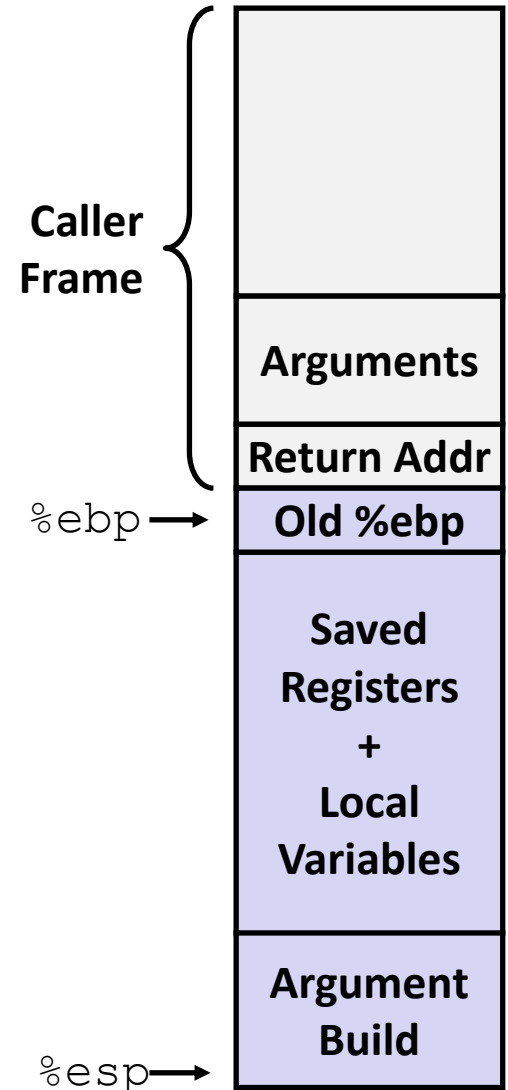
- Stack is the right data structure for procedure call / return
 - If P calls Q, then Q returns before P

■ Recursion (& mutual recursion) handled by normal calling conventions

- Can safely store values in local stack frame and in callee-saved registers
- Put function arguments at top of stack
- Result return in `%eax`

■ Pointers are addresses of values

- On stack or global



x86-64 Integer Registers:

Usage Conventions

%rax	Return value
%rbx	Callee saved
%rcx	Argument #4
%rdx	Argument #3
%rsi	Argument #2
%rdi	Argument #1
%rsp	Stack pointer
%rbp	Callee saved

%r8	Argument #5
%r9	Argument #6
%r10	Caller saved
%r11	Caller Saved
%r12	Callee saved
%r13	Callee saved
%r14	Callee saved
%r15	Callee saved

x86-64 Registers

- **Arguments passed to functions via registers**
 - If more than 6 integral parameters, then pass rest on stack
 - These registers can be used as caller-saved as well
- **All references to stack frame via stack pointer**
 - Eliminates need to update `%ebp/%rbp`
- **Other Registers**
 - 6 callee saved
 - 2 caller saved
 - 1 return value (also usable as caller saved)
 - 1 special (stack pointer)

x86-64 Long Swap

```
void swap_l(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
swap:
    movq    (%rdi), %rdx
    movq    (%rsi), %rax
    movq    %rax, (%rdi)
    movq    %rdx, (%rsi)
    ret
```

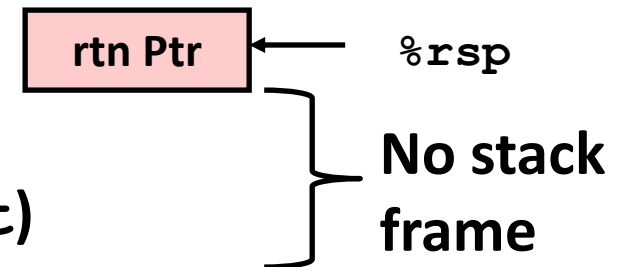
■ Operands passed in registers

- First (**x**p) in %rdi, second (**y**p) in %rsi
- 64-bit pointers

■ No stack operations required (except ret)

■ Avoiding stack

- Can hold all local information in registers



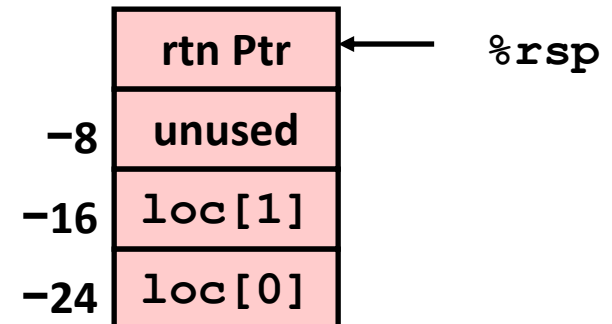
x86-64 Locals in the Red Zone

```
/* Swap, using local array */
void swap_a(long *xp, long *yp)
{
    volatile long loc[2];
    loc[0] = *xp;
    loc[1] = *yp;
    *xp = loc[1];
    *yp = loc[0];
}
```

```
swap_a:
    movq    (%rdi), %rax
    movq    %rax, -24(%rsp)
    movq    (%rsi), %rax
    movq    %rax, -16(%rsp)
    movq    -16(%rsp), %rax
    movq    %rax, (%rdi)
    movq    -24(%rsp), %rax
    movq    %rax, (%rsi)
    ret
```

■ Avoiding Stack Pointer Change

- Can hold all information within small window beyond stack pointer



x86-64 NonLeaf without Stack Frame

```
/* Swap a[i] & a[i+1] */  
void swap_ele(long a[], int i)  
{  
    swap(&a[i], &a[i+1]);  
}
```

- No values held while swap being invoked
- No callee save registers needed
- rep instruction inserted as no-op
 - Based on recommendation from AMD

```
swap_ele:  
    movslq %esi,%rsi          # Sign extend i  
    leaq    8(%rdi,%rsi,8), %rax # &a[i+1]  
    leaq    (%rdi,%rsi,8), %rdi  # &a[i] (1st arg)  
    movq    %rax, %rsi          # (2nd arg)  
    call    swap  
    rep  
    ret                       # No-op
```

x86-64 Stack Frame Example

```
long sum = 0;
/* Swap a[i] & a[i+1] */
void swap_ele_su
    (long a[], int i)
{
    swap(&a[i], &a[i+1]);
    sum += (a[i]*a[i+1]);
}
```

- Keeps values of `&a[i]` and `&a[i+1]` in callee save registers
- Must set up stack frame to save these registers

```
swap_ele_su:
    movq    %rbx, -16(%rsp)
    movq    %rbp, -8(%rsp)
    subq    $16, %rsp
    movslq   %esi, %rax
    leaq    8(%rdi, %rax, 8), %rbx
    leaq    (%rdi, %rax, 8), %rbp
    movq    %rbx, %rsi
    movq    %rbp, %rdi
    call    swap
    movq    (%rbx), %rax
    imulq   (%rbp), %rax
    addq    %rax, sum(%rip)
    movq    (%rsp), %rbx
    movq    8(%rsp), %rbp
    addq    $16, %rsp
    ret
```

Understanding x86-64 Stack Frame

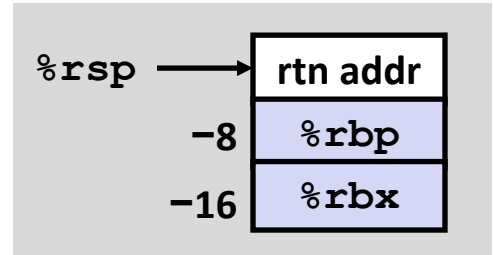
swap_ele_su:

movq	%rbx, -16(%rsp)	# Save %rbx
movq	%rbp, -8(%rsp)	# Save %rbp
subq	\$16, %rsp	# Allocate stack frame
movslq	%esi, %rax	# Extend i
leaq	8(%rdi, %rax, 8), %rbx	# &a[i+1] (callee save)
leaq	(%rdi, %rax, 8), %rbp	# &a[i] (callee save)
movq	%rbx, %rsi	# 2 nd argument
movq	%rbp, %rdi	# 1 st argument
call	swap	
movq	(%rbx), %rax	# Get a[i+1]
imulq	(%rbp), %rax	# Multiply by a[i]
addq	%rax, sum(%rip)	# Add to sum
movq	(%rsp), %rbx	# Restore %rbx
movq	8(%rsp), %rbp	# Restore %rbp
addq	\$16, %rsp	# Deallocate frame
ret		

Understanding x86-64 Stack Frame

```
movq    %rbx, -16(%rsp)
movq    %rbp, -8(%rsp)
```

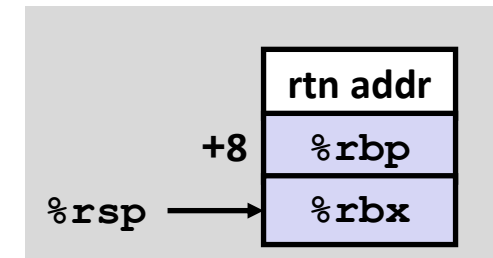
```
# Save %rbx
# Save %rbp
```



```
subq    $16, %rsp
```

```
# Allocate stack frame
```

● ● ●



```
movq    (%rsp), %rbx
movq    8(%rsp), %rbp
```

```
# Restore %rbx
# Restore %rbp
```

```
addq    $16, %rsp
```

```
# Deallocate frame
```

Interesting Features of Stack Frame

■ Allocate entire frame at once

- All stack accesses can be relative to `%rsp`
- Do by decrementing stack pointer
- Can delay allocation, since safe to temporarily use red zone

■ Simple deallocation

- Increment stack pointer
- No base/frame pointer needed

x86-64 Procedure Summary

■ Heavy use of registers

- Parameter passing
- More temporaries since more registers

■ Minimal use of stack

- Sometimes none
- Allocate/deallocate entire block

■ Many tricky optimizations

- What kind of stack frame to use
- Various allocation techniques

Basic Data Types

■ Integral

- Stored & operated on in general (integer) registers
- Signed vs. unsigned depends on instructions used

Intel	ASM	Bytes	C
byte	b	1	[unsigned] char
word	w	2	[unsigned] short
double word	d	4	[unsigned] int
quad word	q	8	[unsigned] long int (x86-64)

■ Floating Point

- Stored & operated on in floating point registers

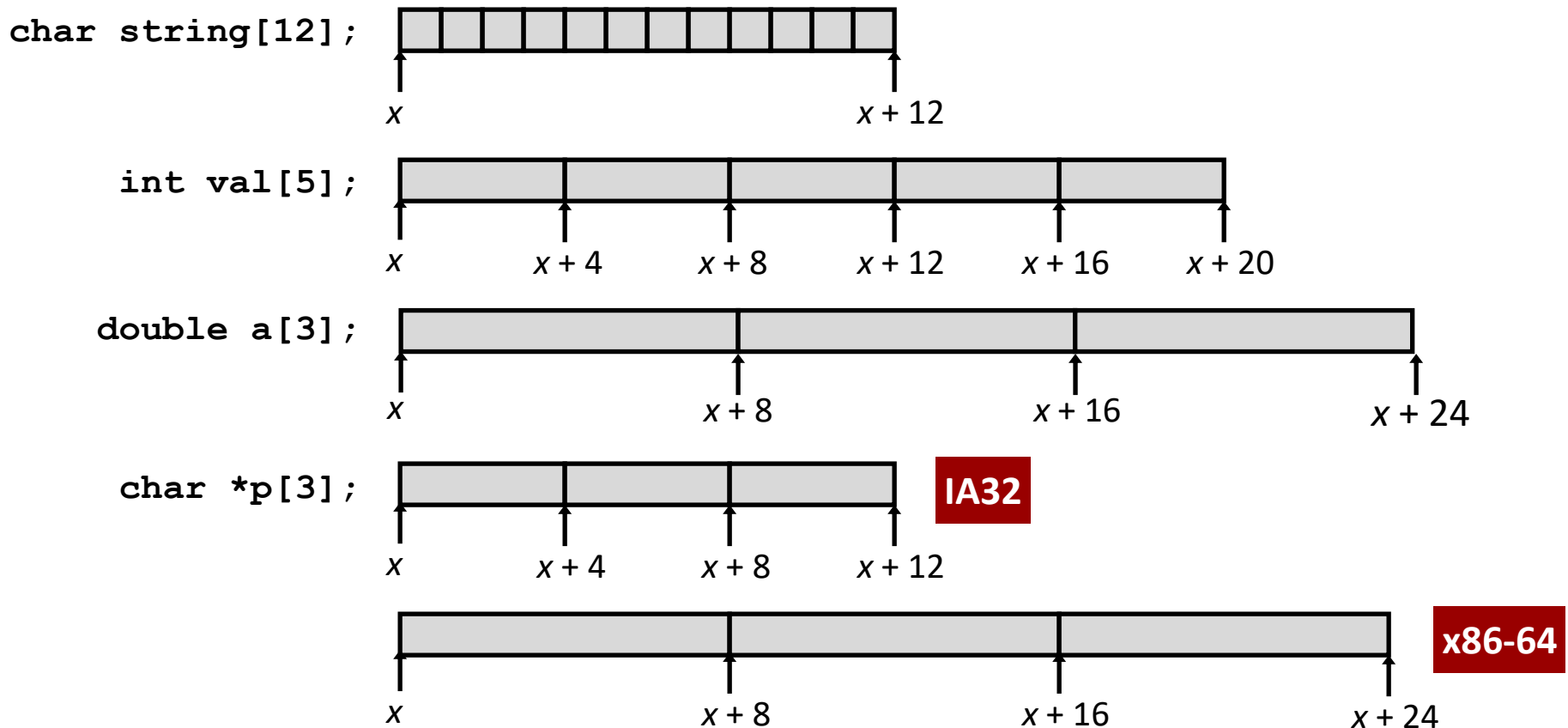
Intel	ASM	Bytes	C
Single	s	4	float
Double	d	8	double
Extended	t	10/12/16	long double

Array Allocation

■ Basic Principle

$T \ A[L];$

- Array of data type T and length L
- Contiguously allocated region of $L * \text{sizeof}(T)$ bytes

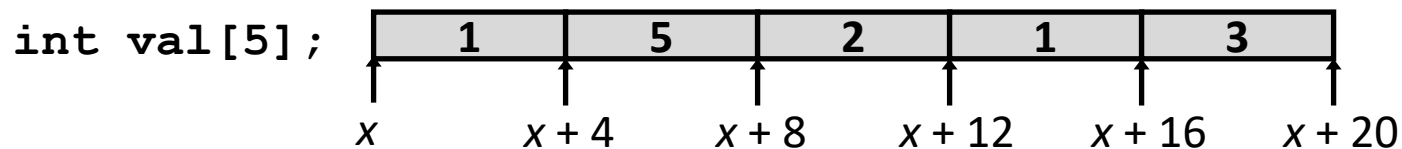


Array Access

■ Basic Principle

T **A**[L] ;

- Array of data type T and length L
- Identifier **A** can be used as a pointer to array element 0: Type T^*

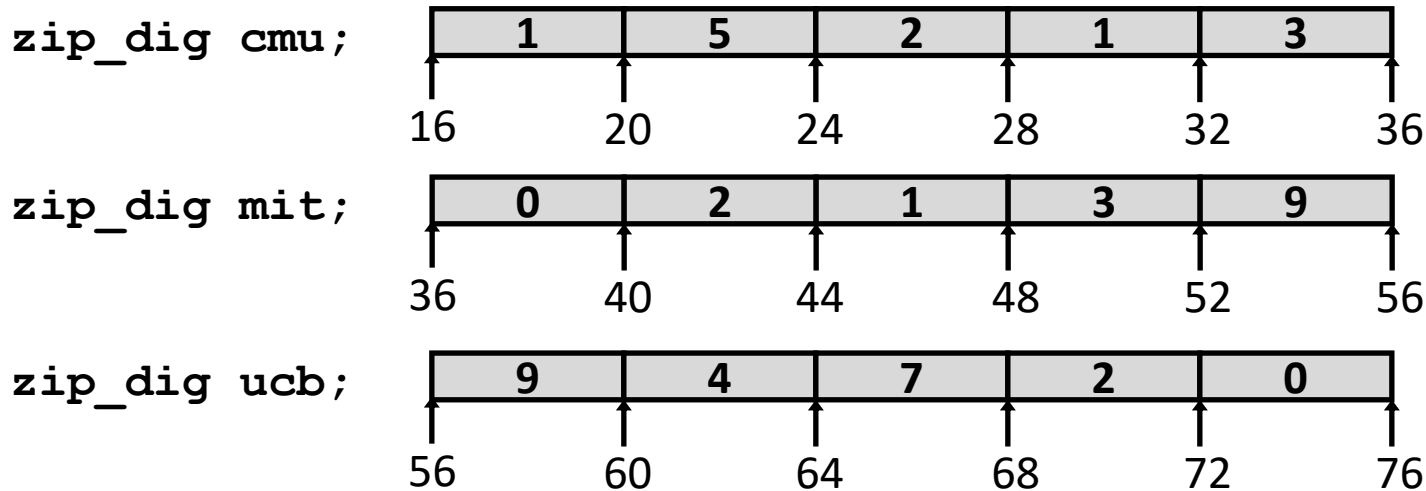


■ Reference	Type	Value
<code>val[4]</code>	<code>int</code>	3
<code>val</code>	<code>int *</code>	x
<code>val+1</code>	<code>int *</code>	$x+4$
<code>&val[2]</code>	<code>int *</code>	$x+8$
<code>val[5]</code>	<code>int</code>	??
<code>*(val+1)</code>	<code>int</code>	5
<code>val + i</code>	<code>int *</code>	$x+4\ i$

Array Example

```
#define ZLEN 5
typedef int zip_dig[ZLEN];

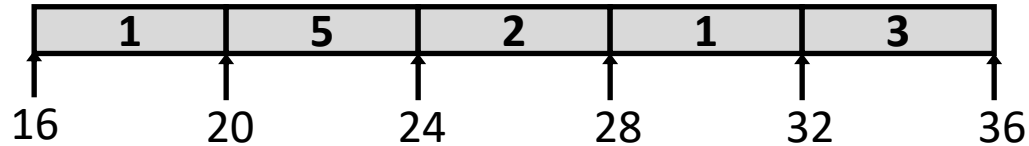
zip_dig cmu = { 1, 5, 2, 1, 3 };
zip_dig mit = { 0, 2, 1, 3, 9 };
zip_dig ucb = { 9, 4, 7, 2, 0 };
```



- Declaration “`zip_dig cmu`” equivalent to “`int cmu[5]`”
- Example arrays were allocated in successive 20 byte blocks
 - Not guaranteed to happen in general

Array Accessing Example

zip_dig cmu;



```
int get_digit
    (zip_dig z, int dig)
{
    return z[dig];
}
```

IA32

```
# %edx = z
# %eax = dig
movl (%edx,%eax,4),%eax # z[dig]
```

- Register `%edx` contains starting address of array
- Register `%eax` contains array index
- Desired digit at $4 * \%eax + \%edx$
- Use memory reference `(%edx, %eax, 4)`

Array Loop Example (IA32)

```
void zincr(zip_dig z) {  
    int i;  
    for (i = 0; i < ZLEN; i++)  
        z[i]++;  
}
```

```
# edx = z  
movl    $0, %eax          # %eax = i  
.L4:      # loop:  
addl    $1, (%edx,%eax,4)  # z[i]++  
addl    $1, %eax          # i++  
cmpl    $5, %eax          # i:5  
jne     .L4               # if !=, goto loop
```

Pointer Loop Example (IA32)

```
void zincr_p(zip_dig z) {  
    int *zend = z+ZLEN;  
    do {  
        (*z)++;  
        z++;  
    } while (z != zend);  
}
```

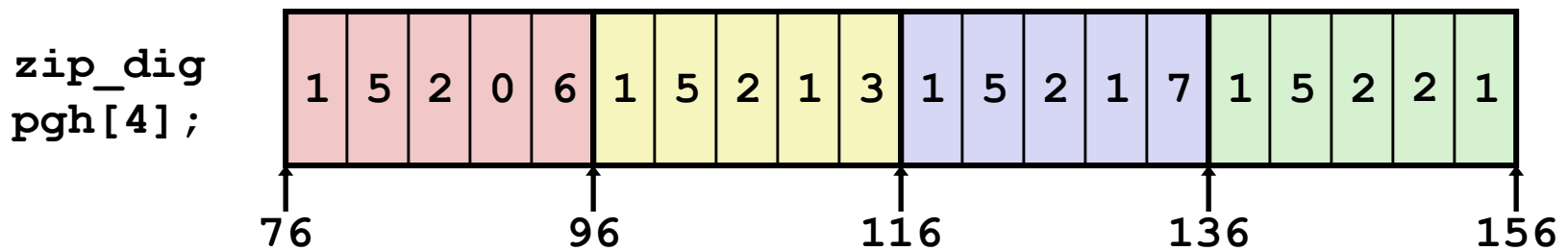


```
void zincr_v(zip_dig z) {  
    void *vz = z;  
    int i = 0;  
    do {  
        (*((int *) (vz+i)))++;  
        i += ISIZE;  
    } while (i != ISIZE*ZLEN);  
}
```

```
# edx = z = vz  
movl    $0, %eax  
.L8:  
addl    $1, (%edx,%eax)  
addl    $4, %eax  
cmpl    $20, %eax  
jne     .L8  
# i = 0  
# loop:  
# Increment vz+i  
# i += 4  
# Compare i:20  
# if !=, goto loop
```


Nested Array Example

```
#define PCOUNT 4
zip_dig pgh[PCOUNT] =
    {{1, 5, 2, 0, 6},
     {1, 5, 2, 1, 3 },
     {1, 5, 2, 1, 7 },
     {1, 5, 2, 2, 1 }};
```



- **“zip_dig pgh[4]” equivalent to “int pgh[4][5]”**
 - Variable **pgh**: array of 4 elements, allocated contiguously
 - Each element is an array of 5 **int**’s, allocated contiguously
- **“Row-Major” ordering of all elements guaranteed**

Multidimensional (Nested) Arrays

■ Declaration

$T \ A[R][C];$

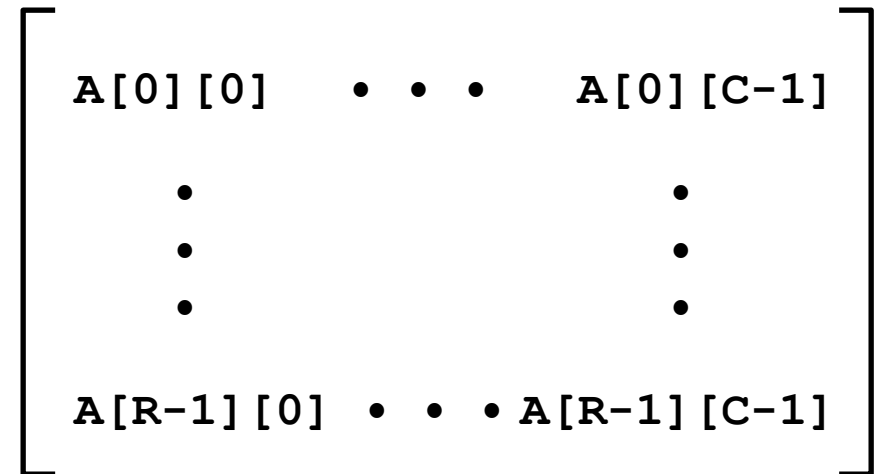
- 2D array of data type T
- R rows, C columns
- Type T element requires K bytes

■ Array Size

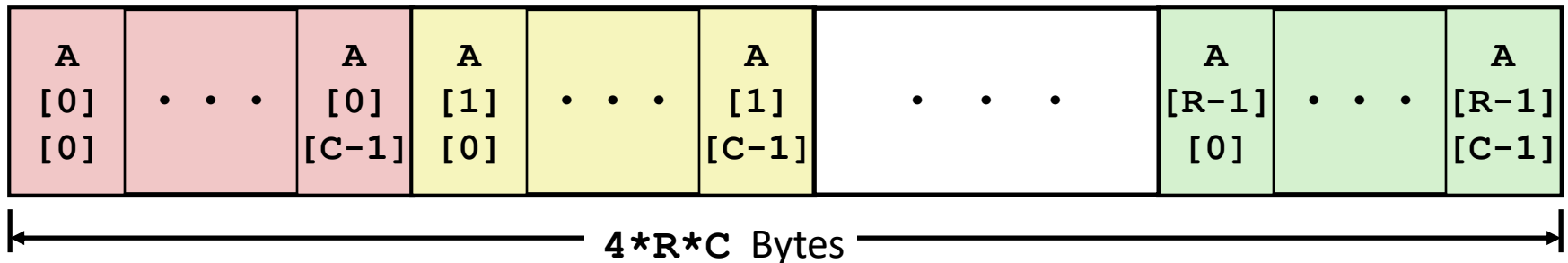
- $R * C * K$ bytes

■ Arrangement

- Row-Major Ordering



`int A[R][C];`

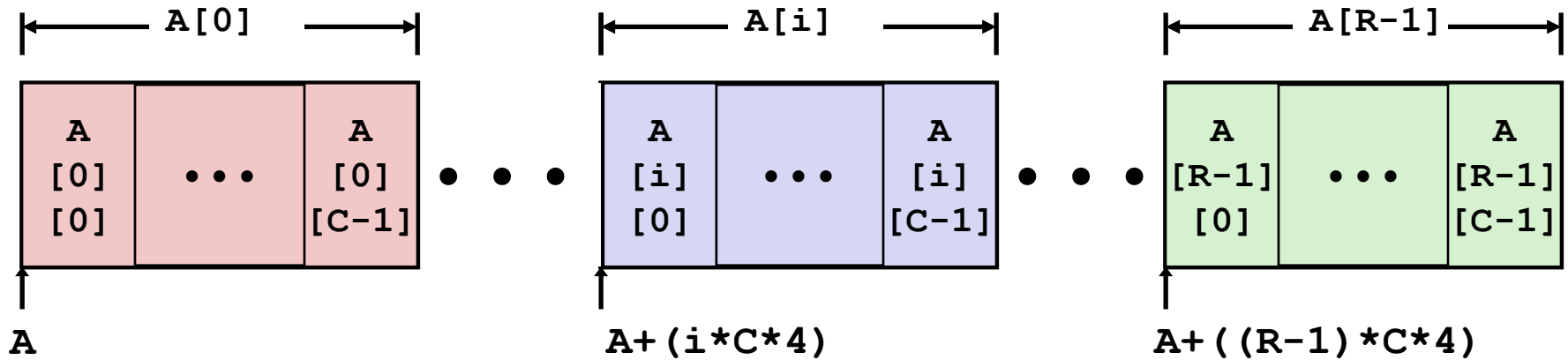


Nested Array Row Access

■ Row Vectors

- $A[i]$ is array of C elements
- Each element of type T requires K bytes
- Starting address $A + i * (C * K)$

```
int A[R][C];
```



Nested Array Row Access Code

```
int *get_pgh_zip(int index)
{
    return pgh[index];
}
```

```
#define PCOUNT 4
zip_dig pgh[PCOUNT] =
    {{1, 5, 2, 0, 6},
     {1, 5, 2, 1, 3 },
     {1, 5, 2, 1, 7 },
     {1, 5, 2, 2, 1 }};
```

```
# %eax = index
leal (%eax,%eax,4),%eax # 5 * index
leal pgh(,%eax,4),%eax  # pgh + (20 * index)
```

■ Row Vector

- `pgh[index]` is array of 5 `int`'s
- Starting address `pgh + (20*index)`

■ IA32 Code

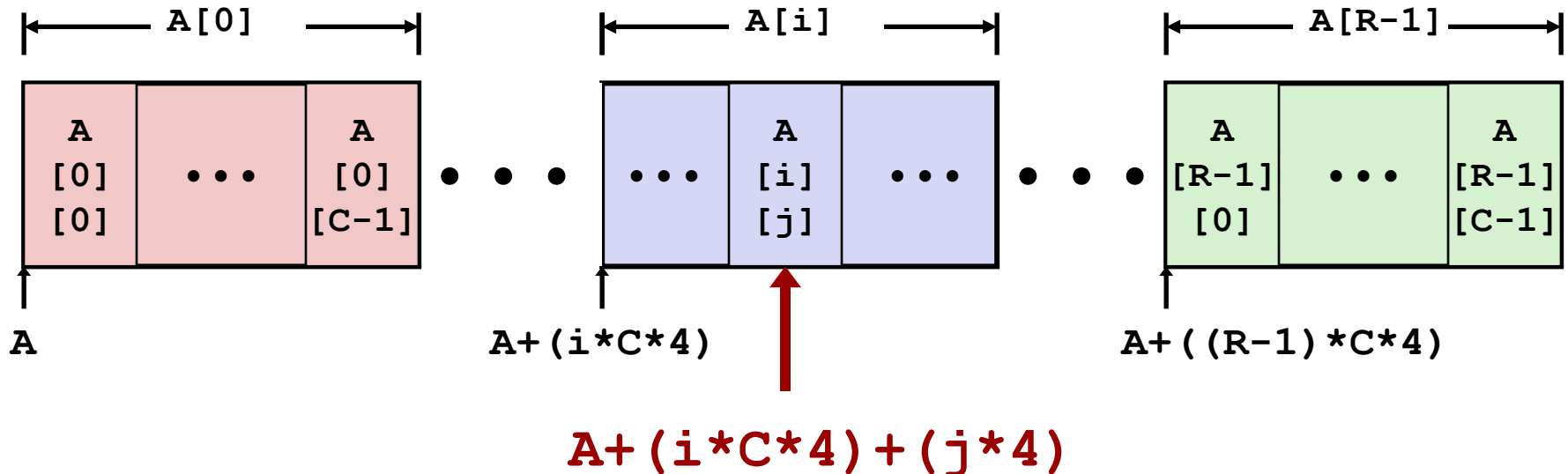
- Computes and returns address
- Compute as `pgh + 4*(index+4*index)`

Nested Array Row Access

■ Array Elements

- $A[i][j]$ is element of type T , which requires K bytes
- Address $A + i * (C * K) + j * K = A + (i * C + j) * K$

```
int A[R][C];
```



Nested Array Element Access Code

```
int get_pgh_digit
(int index, int dig)
{
    return pgh[index][dig];
}
```

```
movl    8(%ebp), %eax        # index
leal    (%eax,%eax,4), %eax   # 5*index
addl    12(%ebp), %eax        # 5*index+dig
movl    pgh(,%eax,4), %eax    # offset 4*(5*index+dig)
```

■ Array Elements

- `pgh[index][dig]` is `int`
- Address: `pgh + 20*index + 4*dig`
 - `= pgh + 4*(5*index + dig)`

■ IA32 Code

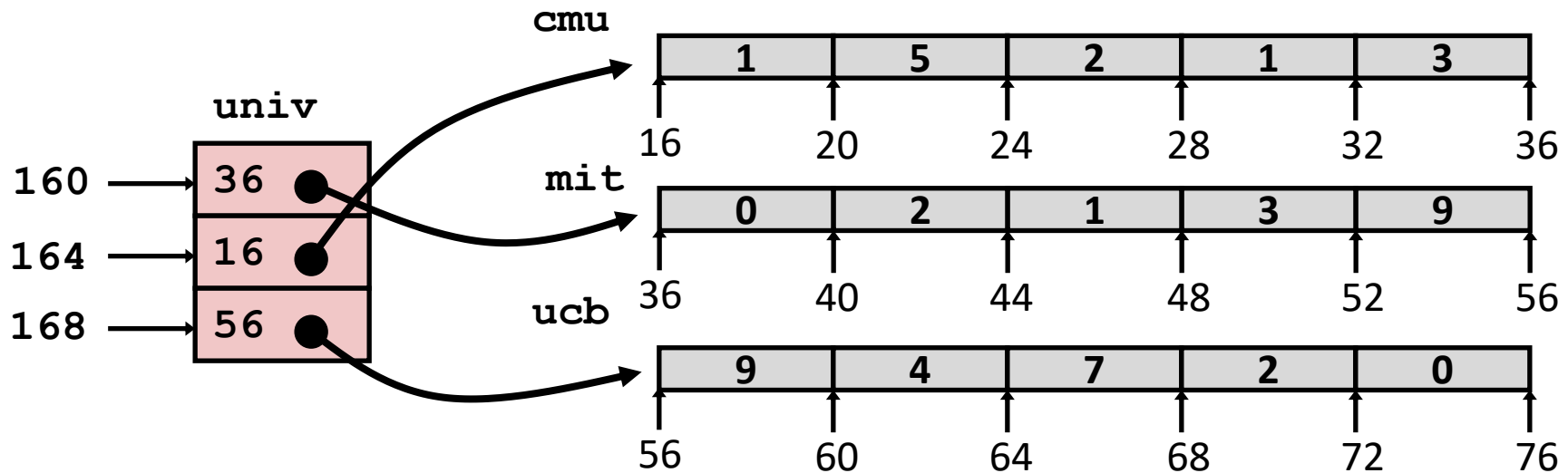
- Computes address `pgh + 4*((index+4*index)+dig)`

Multi-Level Array Example

```
zip_dig cmu = { 1, 5, 2, 1, 3 };  
zip_dig mit = { 0, 2, 1, 3, 9 };  
zip_dig ucb = { 9, 4, 7, 2, 0 };
```

```
#define UCOUNT 3  
int *univ[UCOUNT] = {mit, cmu, ucb};
```

- Variable `univ` denotes array of 3 elements
- Each element is a pointer
 - 4 bytes
- Each pointer points to array of `int`'s



Element Access in Multi-Level Array

```
int get_univ_digit
(int index, int dig)
{
    return univ[index][dig];
}
```

```
movl    8(%ebp), %eax          # index
movl    univ(,%eax,4), %edx     # p = univ[index]
movl    12(%ebp), %eax         # dig
movl    (%edx,%eax,4), %eax     # p[dig]
```

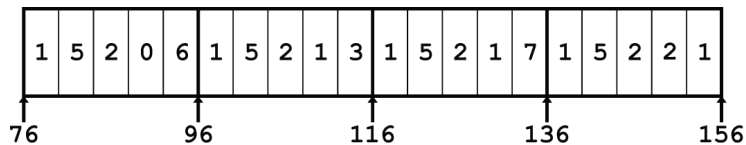
■ Computation (IA32)

- Element access **Mem[Mem[univ+4*index]+4*dig]**
- Must do two memory reads
 - First get pointer to row array
 - Then access element within array

Array Element Accesses

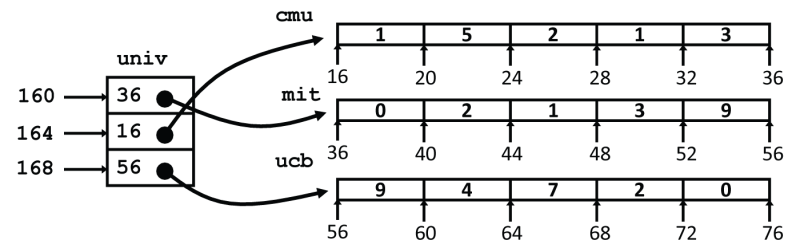
Nested array

```
int get_pgh_digit
(int index, int dig)
{
    return pgh[index][dig];
}
```



Multi-level array

```
int get_univ_digit
(int index, int dig)
{
    return univ[index][dig];
}
```



Accesses looks similar in C, but addresses very different:

$\text{Mem}[\text{pgh} + 20 * \text{index} + 4 * \text{dig}]$

$\text{Mem}[\text{Mem}[\text{univ} + 4 * \text{index}] + 4 * \text{dig}]$

N X N Matrix Code

■ Fixed dimensions

- Know value of N at compile time

```
#define N 16
typedef int fix_matrix[N][N];
/* Get element a[i][j] */
int fix_ele
    (fix_matrix a, int i, int j)
{
    return a[i][j];
}
```

■ Variable dimensions, explicit indexing

- Traditional way to implement dynamic arrays

```
#define IDX(n, i, j) ((i)*(n)+(j))
/* Get element a[i][j] */
int vec_ele
    (int n, int *a, int i, int j)
{
    return a[IDX(n,i,j)];
}
```

■ Variable dimensions, implicit indexing

- Now supported by gcc

```
/* Get element a[i][j] */
int var_ele
    (int n, int a[n][n], int i, int j)
{
    return a[i][j];
}
```

16 X 16 Matrix Access

■ Array Elements

- Address $\mathbf{A} + i * (\mathbf{C} * \mathbf{K}) + j * \mathbf{K}$
- $\mathbf{C} = 16, \mathbf{K} = 4$

```
/* Get element a[i][j] */  
int fix_ele(fix_matrix a, int i, int j) {  
    return a[i][j];  
}
```

```
movl    12(%ebp), %edx    # i  
sall    $6, %edx         # i*64  
movl    16(%ebp), %eax    # j  
sall    $2, %eax         # j*4  
addl    8(%ebp), %eax     # a + j*4  
movl    (%eax,%edx), %eax # *(a + j*4 + i*64)
```

n X n Matrix Access

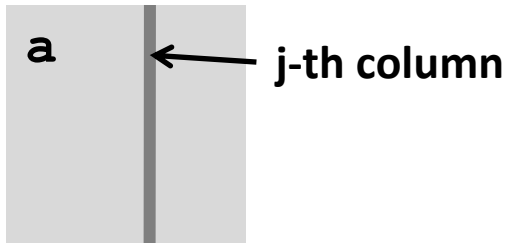
■ Array Elements

- Address $A + i * (C * K) + j * K$
- $C = n, K = 4$

```
/* Get element a[i][j] */  
int var_ele(int n, int a[n][n], int i, int j) {  
    return a[i][j];  
}
```

```
movl    8(%ebp), %eax    # n  
sall    $2, %eax        # n*4  
movl    %eax, %edx      # n*4  
imull   16(%ebp), %edx   # i*n*4  
movl    20(%ebp), %eax   # j  
sall    $2, %eax        # j*4  
addl    12(%ebp), %eax   # a + j*4  
movl    (%eax,%edx), %eax # *(a + j*4 + i*n*4)
```

Optimizing Fixed Array Access



```
#define N 16
typedef int fix_matrix[N][N];
```

■ Computation

- Step through all elements in column j

■ Optimization

- Retrieving successive elements from single column

```
/* Retrieve column j from array */
void fix_column
(fix_matrix a, int j, int *dest)
{
    int i;
    for (i = 0; i < N; i++)
        dest[i] = a[i][j];
}
```

Optimizing Fixed Array Access

■ Optimization

- Compute $\text{ajp} = \&a[i][j]$
 - Initially $= a + 4*j$
 - Increment by $4*N$

Register	Value
%ecx	ajp
%ebx	dest
%edx	i

```
/* Retrieve column j from array */
void fix_column
    (fix_matrix a, int j, int *dest)
{
    int i;
    for (i = 0; i < N; i++)
        dest[i] = a[i][j];
}
```

```
.L8:                                # loop:
    movl    (%ecx), %eax            #   Read *ajp
    movl    %eax, (%ebx,%edx,4)     #   Save in dest[i]
    addl    $1, %edx               #   i++
    addl    $64, %ecx              #   ajp += 4*N
    cmpl    $16, %edx              #   i:N
    jne     .L8                    #   if !=, goto loop
```

Optimizing Variable Array Access

- Compute $ajp = \&a[i][j]$
 - Initially $= a + 4*j$
 - Increment by $4*n$

Register	Value
%ecx	ajp
%edi	dest
%edx	i
%ebx	4*n
%esi	n

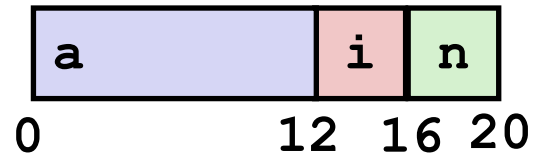
```
/* Retrieve column j from array */
void var_column
(int n, int a[n][n],
 int j, int *dest)
{
    int i;
    for (i = 0; i < n; i++)
        dest[i] = a[i][j];
}
```

```
.L18:                                # loop:
    movl    (%ecx), %eax             #   Read *ajp
    movl    %eax, (%edi,%edx,4)      #   Save in dest[i]
    addl    $1, %edx                #   i++
    addl    $ebx, %ecx               #   ajp += 4*n
    cmpl    $edx, %esi              #   n:i
    jg      .L18                    #   if >, goto loop
```

Structure Allocation

```
struct rec {  
    int a[3];  
    int i;  
    struct rec *n;  
};
```

Memory Layout

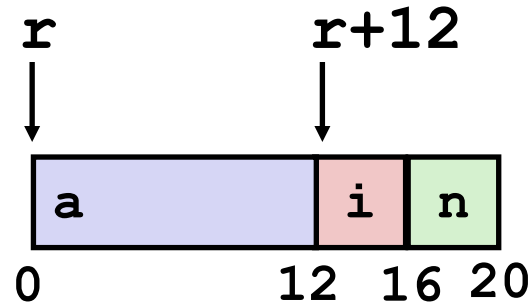


■ Concept

- Contiguously-allocated region of memory
- Refer to members within structure by names
- Members may be of different types

Structure Access

```
struct rec {  
    int a[3];  
    int i;  
    struct rec *n;  
};
```



■ Accessing Structure Member

- Pointer indicates first byte of structure
- Access elements with offsets

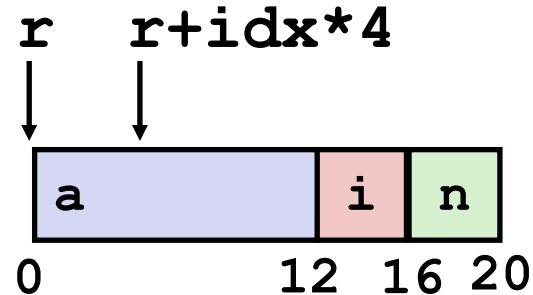
```
void  
set_i(struct rec *r,  
      int val)  
{  
    r->i = val;  
}
```

IA32 Assembly

```
# %edx = val  
# %eax = r  
movl %edx, 12(%eax) # Mem[r+12] = val
```

Generating Pointer to Structure Member

```
struct rec {  
    int a[3];  
    int i;  
    struct rec *n;  
};
```



■ Generating Pointer to Array Element

- Offset of each structure member determined at compile time
- Arguments
 - `Mem[%ebp+8]: r`
 - `Mem[%ebp+12]: idx`

```
int *get_ap  
(struct rec *r, int idx)  
{  
    return &r->a[idx];  
}
```

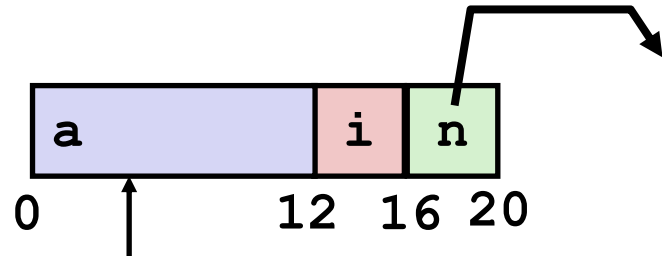
```
movl    12(%ebp), %eax    # Get idx  
sall    $2, %eax         # idx*4  
addl    8(%ebp), %eax     # r+idx*4
```

Following Linked List

■ C Code

```
void set_val
(struct rec *r, int val)
{
    while (r) {
        int i = r->i;
        r->a[i] = val;
        r = r->n;
    }
}
```

```
struct rec {
    int a[3];
    int i;
    struct rec *n;
};
```



Element i

Register	Value
%edx	r
%ecx	val

```
.L17:                                # loop:
    movl    12(%edx), %eax            # r->i
    movl    %ecx, (%edx,%eax,4)      # r->a[i] = val
    movl    16(%edx), %edx           # r = r->n
    testl   %edx, %edx               # Test r
    jne     .L17                     # If != 0 goto loop
```

Summary

■ Procedures in x86-64

- Stack frame is relative to stack pointer
- Parameters passed in registers

■ Arrays

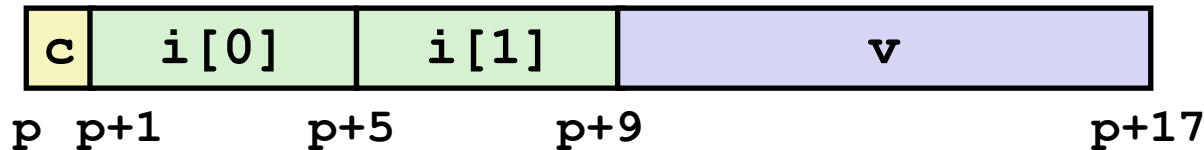
- One-dimensional
- Multi-dimensional (nested)
- Multi-level

■ Structures

- Allocation
- Access

Structures & Alignment

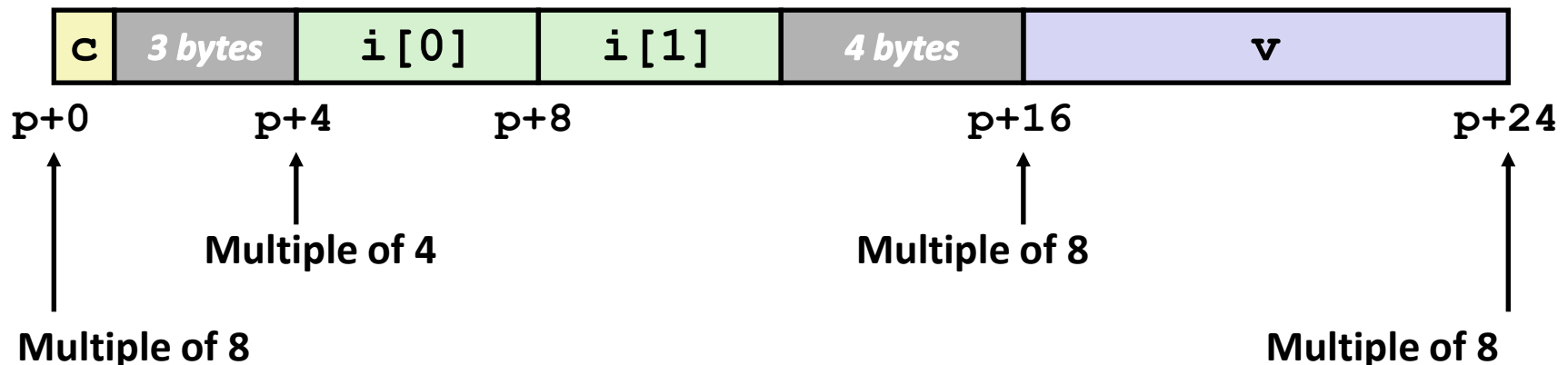
■ Unaligned Data



```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```

■ Aligned Data

- Primitive data type requires K bytes
- Address must be multiple of K



Alignment Principles

■ Aligned Data

- Primitive data type requires K bytes
- Address must be multiple of K
- Required on some machines; advised on IA32
 - treated differently by IA32 Linux, x86-64 Linux, and Windows!

■ Motivation for Aligning Data

- Memory accessed by (aligned) chunks of 4 or 8 bytes (system dependent)
 - Inefficient to load or store datum that spans quad word boundaries
 - Virtual memory very tricky when datum spans 2 pages

■ Compiler

- Inserts gaps in structure to ensure correct alignment of fields

Specific Cases of Alignment (IA32)

- **1 byte: `char`, ...**
 - no restrictions on address
- **2 bytes: `short`, ...**
 - lowest 1 bit of address must be 0_2
- **4 bytes: `int`, `float`, `char *`, ...**
 - lowest 2 bits of address must be 00_2
- **8 bytes: `double`, ...**
 - Windows (and most other OS's & instruction sets):
 - lowest 3 bits of address must be 000_2
 - Linux:
 - lowest 2 bits of address must be 00_2
 - i.e., treated the same as a 4-byte primitive data type
- **12 bytes: `long double`**
 - Windows, Linux:
 - lowest 2 bits of address must be 00_2
 - i.e., treated the same as a 4-byte primitive data type

Specific Cases of Alignment (x86-64)

- **1 byte: char, ...**
 - no restrictions on address
- **2 bytes: short, ...**
 - lowest 1 bit of address must be 0_2
- **4 bytes: int, float, ...**
 - lowest 2 bits of address must be 00_2
- **8 bytes: double, char *, ...**
 - Windows & Linux:
 - lowest 3 bits of address must be 000_2
- **16 bytes: long double**
 - Linux, Windows:
 - lowest 3 bits of address must be 000_2
 - i.e., treated the same as a 8-byte primitive data type

Satisfying Alignment with Structures

■ Within structure:

- Must satisfy each element's alignment requirement

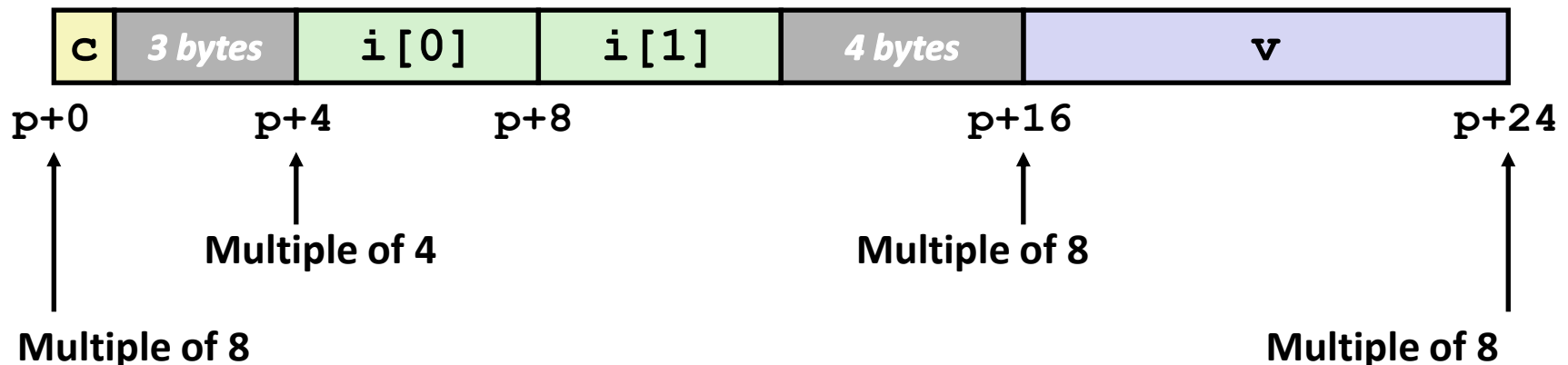
■ Overall structure placement

- Each structure has alignment requirement K
 - K = Largest alignment of any element
- Initial address & structure length must be multiples of K

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```

■ Example (under Windows or x86-64):

- $K = 8$, due to `double` element

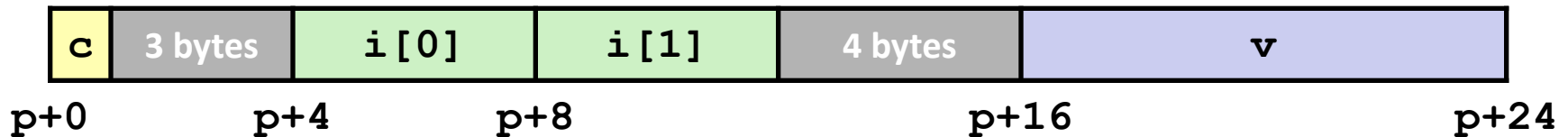


Different Alignment Conventions

■ x86-64 or IA32 Windows:

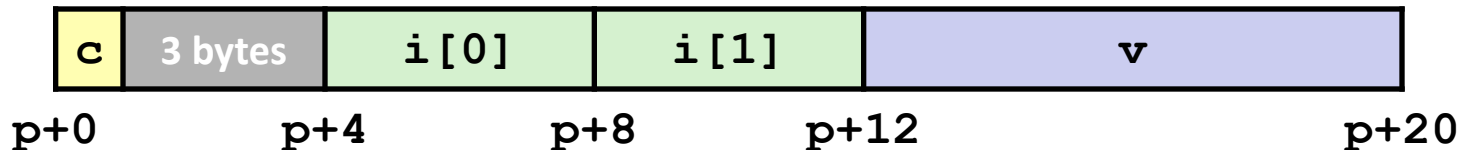
- $K = 8$, due to `double` element

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```



■ IA32 Linux

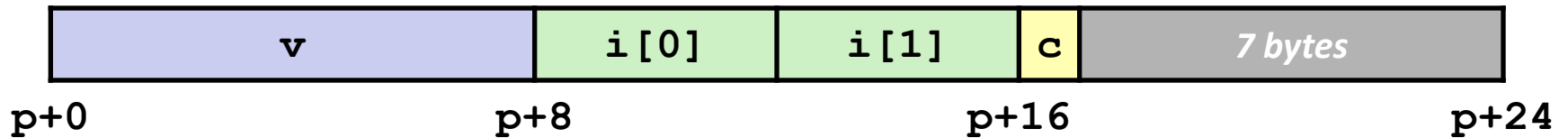
- $K = 4$; `double` treated like a 4-byte data type



Meeting Overall Alignment Requirement

- For largest alignment requirement K
- Overall structure must be multiple of K

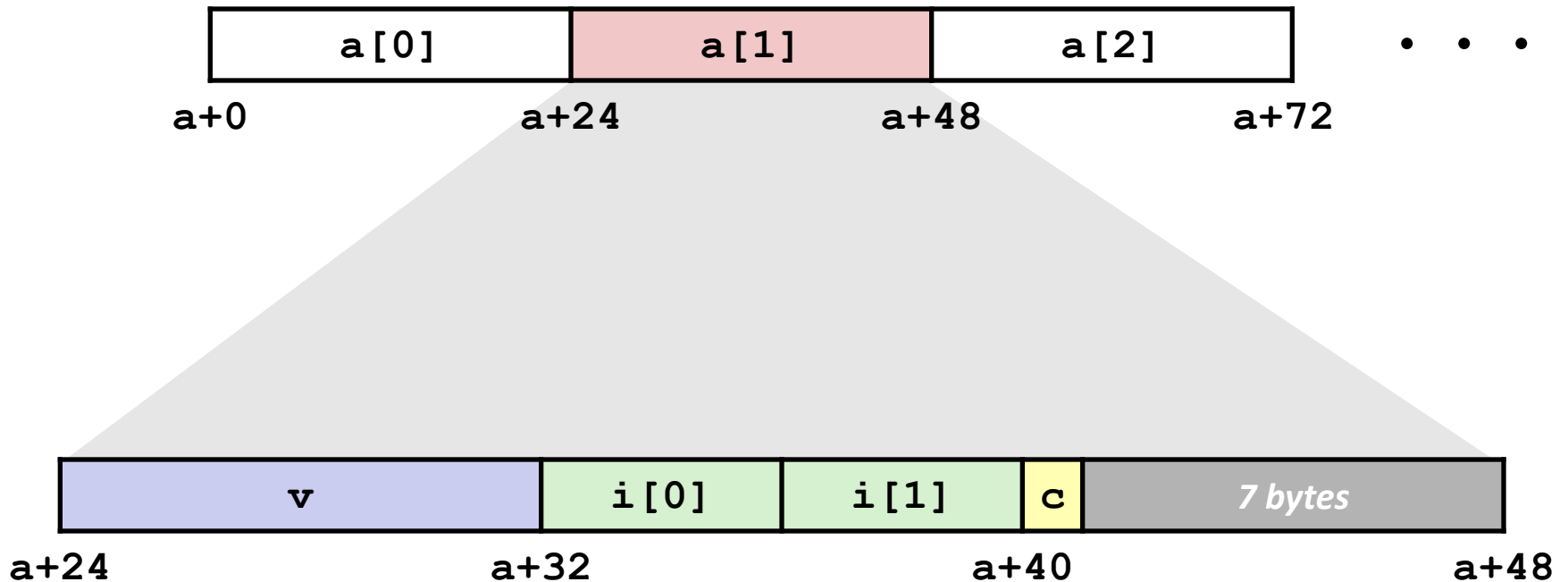
```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} *p;
```



Arrays of Structures

- Overall structure length multiple of K
- Satisfy alignment requirement for every element

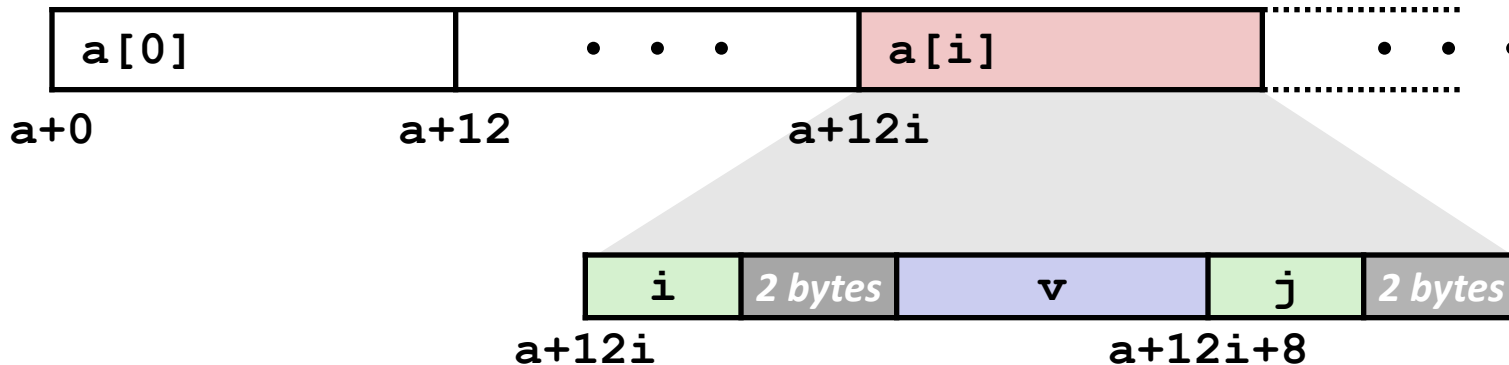
```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} a[10];
```



Accessing Array Elements

- Compute array offset $12i$
 - `sizeof(S3)`, including alignment spacers
- Element `j` is at offset 8 within structure
- Assembler gives offset `a+8`
 - Resolved during linking

```
struct S3 {  
    short i;  
    float v;  
    short j;  
} a[10];
```



```
short get_j(int idx)  
{  
    return a[idx].j;  
}
```

```
# %eax = idx  
leal (%eax,%eax,2),%eax # 3*idx  
movswl a+8(,%eax,4),%eax
```

Saving Space

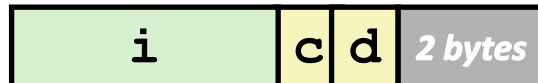
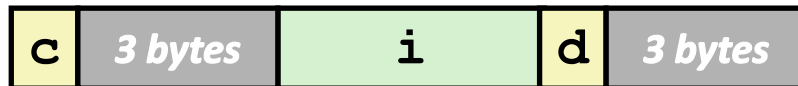
- Put large data types first

```
struct S4 {  
    char c;  
    int i;  
    char d;  
} *p;
```



```
struct S5 {  
    int i;  
    char c;  
    char d;  
} *p;
```

- Effect (K=4)

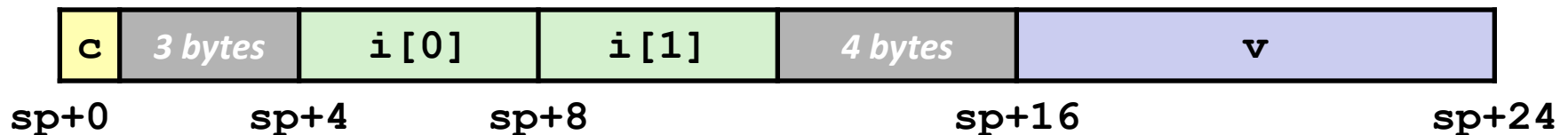
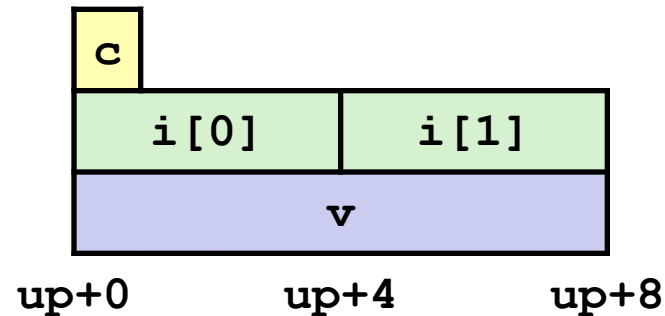


Union Allocation

- Allocate according to largest element
- Can only use one field at a time

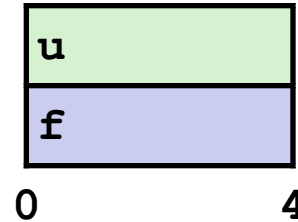
```
union U1 {  
    char c;  
    int i[2];  
    double v;  
} *up;
```

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *sp;
```



Using Union to Access Bit Patterns

```
typedef union {  
    float f;  
    unsigned u;  
} bit_float_t;
```



```
float bit2float(unsigned u)  
{  
    bit_float_t arg;  
    arg.u = u;  
    return arg.f;  
}
```

Same as (float) u ?

```
unsigned float2bit(float f)  
{  
    bit_float_t arg;  
    arg.f = f;  
    return arg.u;  
}
```

Same as (unsigned) f ?